



Context API





**Activen las cámaras los que puedan y
pasemos asistencia**

***Gestionar el estado global
de una SPA utilizando
Context API para compartir
datos entre componentes y
vistas de una aplicación
React.***

{desafío}
latam_

- Unidad 1: Introducción a React.
- Unidad 2: Estados de los componentes y eventos.
- Unidad 3: Renderización dinámica de componentes.
- Unidad 4: Consumo de APIs con React.
- Unidad 5: React Router I
- Unidad 6: Context
- Unidad 7: React Router II
- Unidad 8: JWT



Te encuentras aquí



Inicio

{desafío}
latam_



Activación de conceptos

Contesta la pregunta correctamente y gana un punto

Instrucciones:

- Se realizará una pregunta, el primero en escribir “YO” por el chat, dará su respuesta al resto de la clase.
- El docente validará la respuesta.
- En caso de que no sea correcta, dará la oportunidad a la segunda persona que dijo “Yo”.
- Cada estudiante podrá participar un máximo de 2 veces.
- Al final, el/la docente indicará el 1º, 2º y 3º lugar.
- Esta actividad no es calificada, es solo una dinámica para recordar los conceptos clave para abordar esta sesión.





Activación de conceptos



¿Qué problema resuelve React Router?



Activación de conceptos



¿Cuál es el rol del componente Browser Route?



Activación de conceptos



¿Cuál es la diferencia entre el componente Link y una etiqueta de hipervínculo?



Activación de conceptos



¿Cómo configuramos una ruta por defecto?



Activación de conceptos



¿Qué es una SPA?



Activación de conceptos



Primer lugar:



Segundo lugar:



Tercer lugar:

*/** Compartir estados entre componentes utilizando props. **/*

*/** Compartir estados entre componentes utilizando Context. **/*

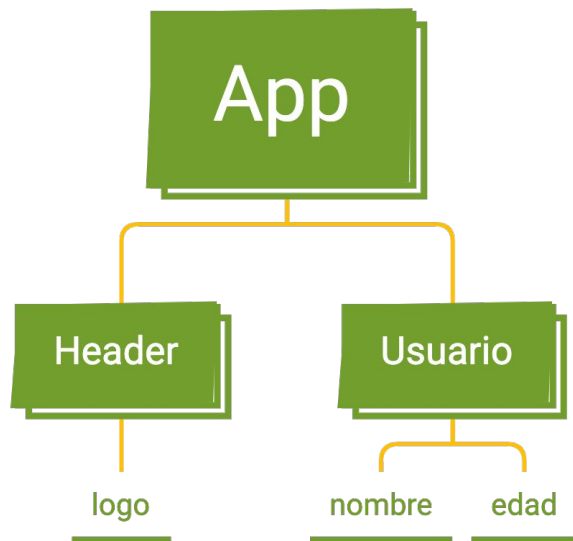
*/** Conocer el flujo de trabajo de React Router con Context para compartir información entre múltiples páginas. **/*

Objetivos

Estados y múltiples componentes

Introducción

Cada componente tiene sus propios estados, estos no se comparten ni podemos acceder directamente a ellos desde otro componente. Es por eso que decimos << **los estados son locales** >>.



- En este diagrama tenemos la componente App, Header y Usuario.
- Header no puede cambiar el estado nombre o el estado edad de un Usuario, por lo menos no directamente.

Estados y múltiples componentes

Veamos un ejemplo con un componente

```
1 import { useState } from "react"
2
3 const Cuenta = () => {
4   const [cuenta, setCuenta] = useState(0)
5
6   return (
7     <div>
8       <ul>
9         {cuenta}
10        <button onClick={() => setCuenta(cuenta + 1)}>+</button>
11      </ul>
12    </div>
13  )
14 }
15
16 export default Cuenta
17
```

Aquí tenemos un componente que muestra el estado cuenta y tiene un botón clic que aumenta la cuenta en 1.



```
1 import { useState } from "react"
2 import Aumentador from "../Aumentador"
3
4 const Cuenta = () => {
5   const [cuenta, setCuenta] = useState(0)
6
7   return (
8     <div>
9       <ul>
10        {cuenta}
11        <Aumentador />
12      </ul>
13    </div>
14  )
15 }
16
17 export default Cuenta
18
19
```

Estados y múltiples componentes

Separando el ejemplo en 2 componentes

Si separamos el botón que aumenta la cuenta en un nuevo componente, no tendremos acceso al estado cuenta ni al setter setCuenta.

```
1 import { useState } from "react"
2
3 const Aumentador = () => {
4
5   return (
6     <button onClick={() => setCuenta(cuenta + 1)}>+</button>
7   )
8 }
9
10 export default Aumentador
```

ERROR

[eslint]

src/Aumentador.jsx

Line 6:28: 'setCuenta' is not defined

no-undef

Line 6:38: 'cuenta' is not defined

no-undef

Estados y múltiples componentes

Pasando el setter como props

```
1 import { useState } from "react"
2 import Aumentador from "./Aumentador"
3
4 const Cuenta = () => {
5   const [cuenta, setCuenta] = useState(0)
6
7   return (
8     <div>
9       <ul>
10        {cuenta}
11        <Aumentador setCuenta={setCuenta} cuenta={cuenta} />
12      </ul>
13    </div>
14  )
15 }
16
17 export default Cuenta
```

Para resolverlo, podemos pasar una función y el estado compartido como prop a la componente hija.

```
1 import { useState } from "react"
2
3 const Aumentador = ({setCuenta, cuenta}) => {
4
5   return (
6     <button onClick={() => setCuenta(cuenta + 1)}>+</button>
7   )
8 }
9
10 export default Aumentador
```


Estados y múltiples componentes

¡Puntos importantes!



- Los estados son locales al componente.
- Podemos compartir un estado con una componente hija pasándolo como prop.

Ejercicio

Repite los pasos:

1. Crea un proyecto nuevo.
2. Crea el componente "Componente1" y agrega el estado cuenta con el valor inicial 10 y con la función setter setCuenta.
3. Crea el componente "Componente2" y muestra el valor de cuenta. Recuerda pasar el valor de cuenta como prop.
4. Bonus: Crea un botón dentro de Componente2 que aumente el valor de cuenta. Si no has pasado como prop la función setCuenta en el paso anterior, ahora tendrás que hacerlo.

Ejercicio ¡Manos al teclado!



*/** Compartir estados entre componentes utilizando props. **/* 

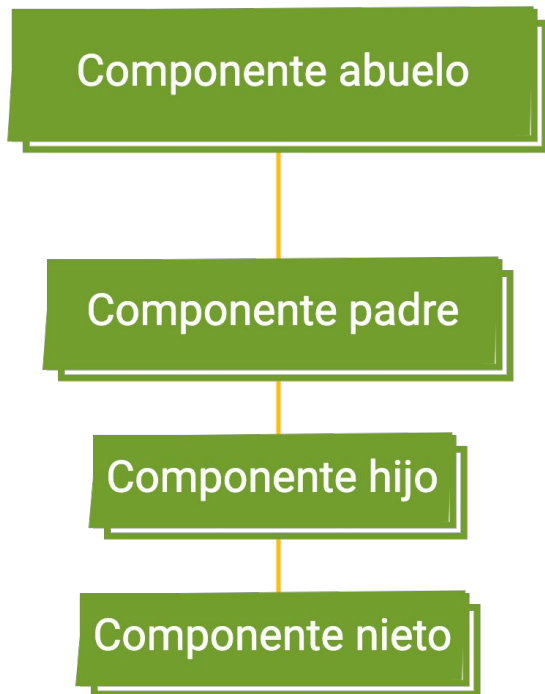
*/** Compartir estados entre componentes utilizando Context. **/*

*/** Conocer el flujo de trabajo de React Router con Context para compartir información entre múltiples páginas. **/*

Objetivos

Estados y múltiples componentes

Prop Drilling

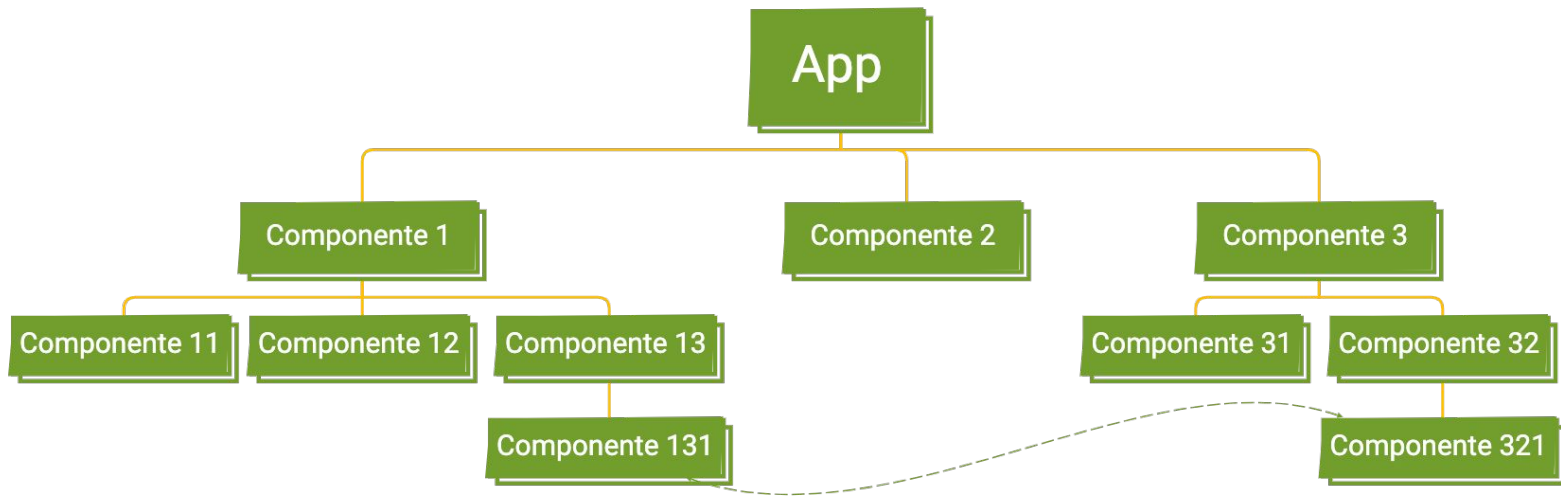


- En algunos casos, para poder separar el código en múltiples componentes tenemos que pasar el estado a lo largo de todas ellas.
- A esto se le llama prop drilling.

Estados y múltiples componentes

Prop Drilling

¿Qué pasaría si nuestra aplicación fuera más compleja?

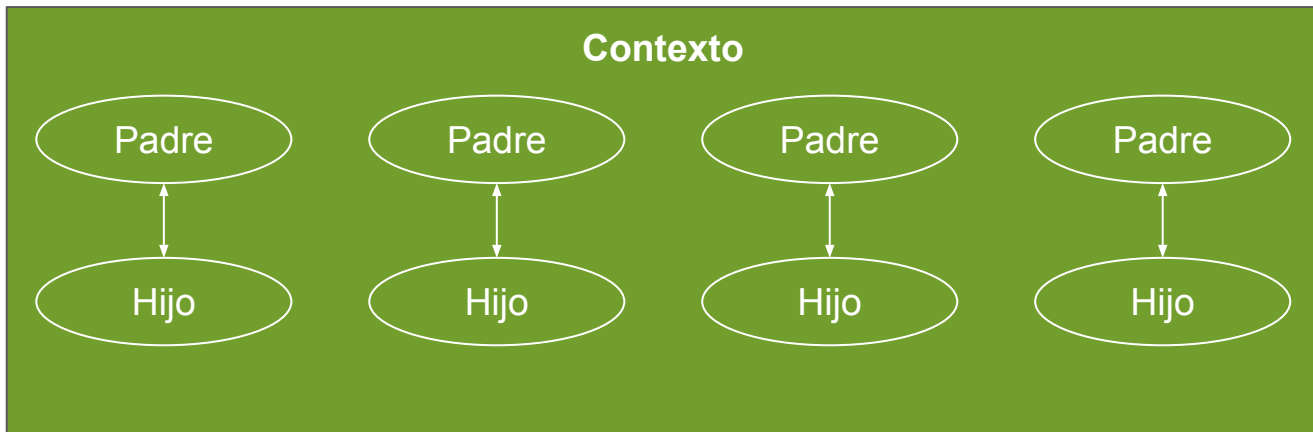


El Prop Drilling complicaría la mantención y escalabilidad de la aplicación.

Estados y múltiples componentes

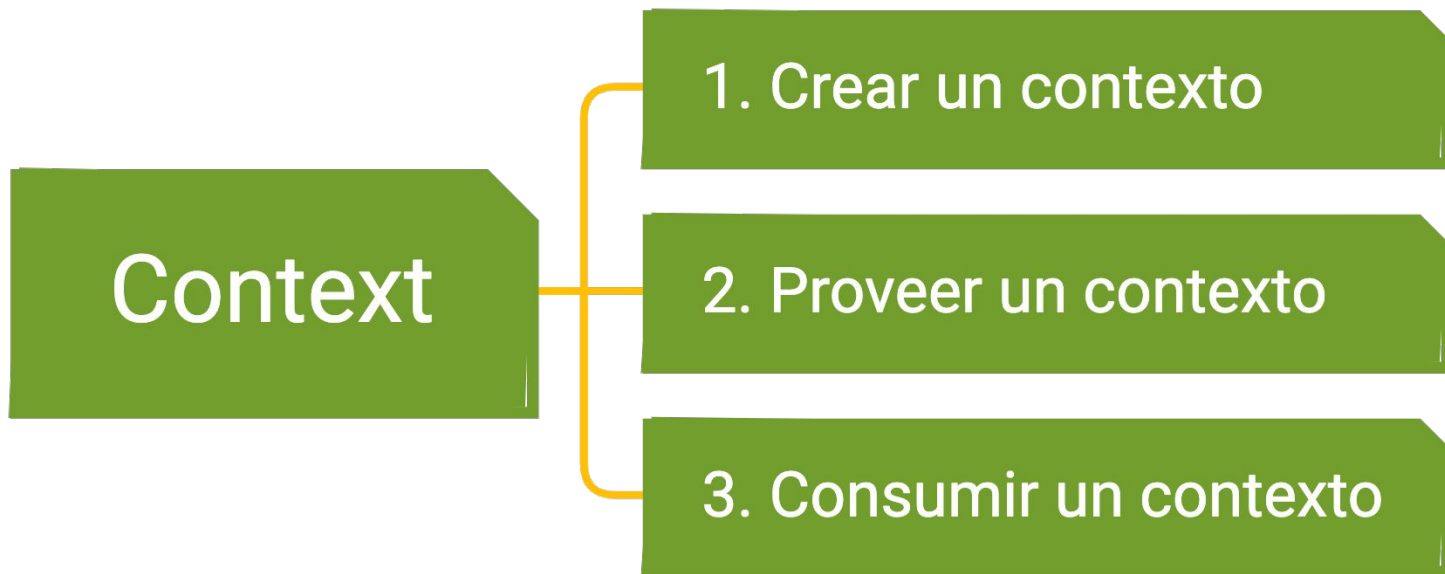
Introducción a Context

En React, podemos ocupar **Context** para crear un estado global o un estado de mayor alcance que un estado local, para compartir estados entre múltiples o incluso todos los componentes.



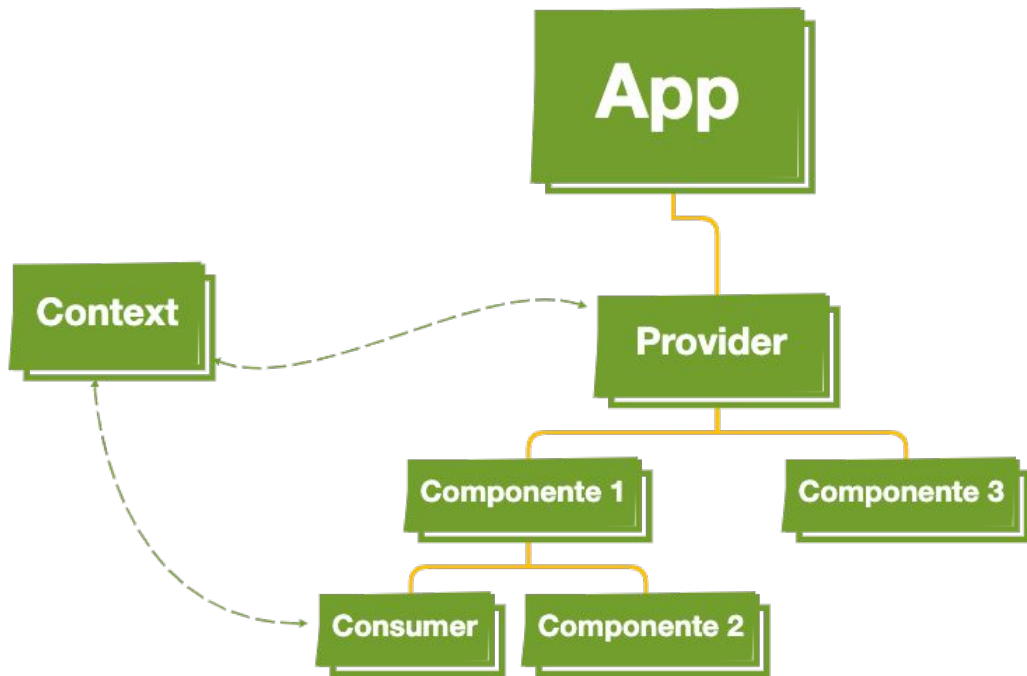
Contexto

Pasos importantes de la implementación de un contexto



Contexto

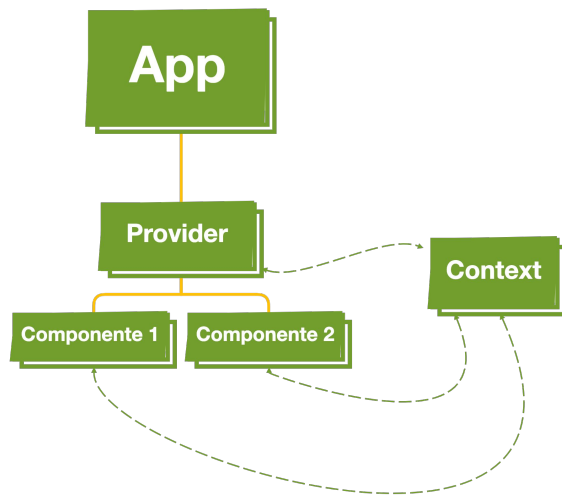
Una aplicación con Context



1. Cada **Context** lo debemos crear como un módulo independiente, que importaremos donde lo necesitemos.
2. Envolveremos todos los componentes que queremos mantener actualizados dentro del componente **Provider**.
3. Cualquier componente puede ser un **Consumer**. Cuando se modifique el valor de un contexto, se volverá a actualizar el consumer.

Realizaremos el ejercicio para mostrar el contador de clics:

- El componente 1 tendrá un botón para aumentar.
- El componente 2 mostrará el valor.
- Utilizaremos un contexto para poder compartir la cuenta y la función setCuenta entre componentes.



Contexto

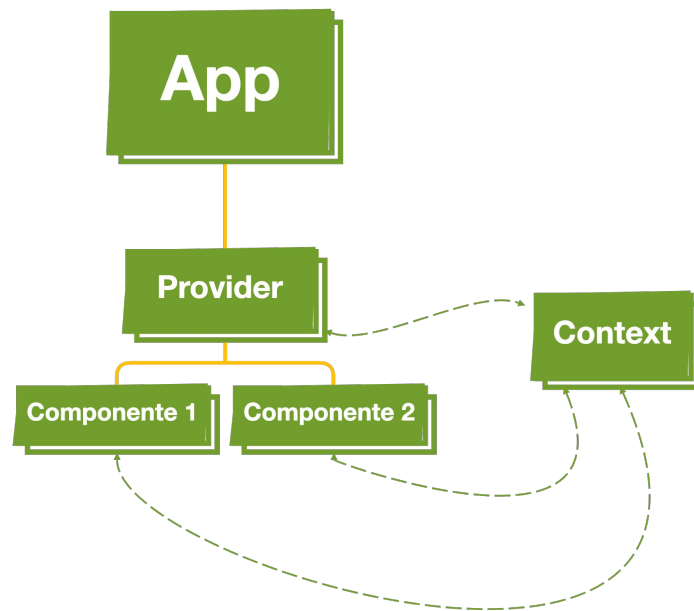
Compartiendo datos con contexto

Contexto

Compartiendo datos con contexto

Setup básico del proyecto:

1. Crearemos el proyecto con `npm vite@latest context-example`
2. Crearemos el componente "Component1".
3. Crearemos el componente "Component2".
4. Importaremos ambos componentes dentro de APP.



Contexto

Compartiendo datos con contexto App.js

```
1  /* App.js */
2
3  import Component1 from "../Component1";
4  import Component2 from "../Component2";
5
6  function App() {
7    return (
8      <>
9        <Component1 />
10       <Component2 />
11     </>
12   );
13 }
14
15 export default App;
16
```

```
1  /* Component1.jsx */
2  const Component1 = () => {
3    return (
4      <p> Hello ! </p>
5    );
6  }
7
8  export default Component1
```

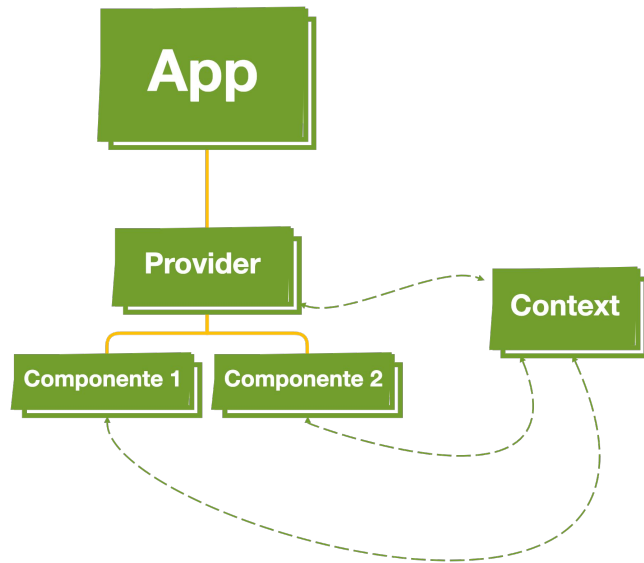
Crear un componente 2 igual a
Componente 1

Contexto

Compartiendo datos con contexto

Ahora bien, para compartir datos utilizando contexto, debemos:

1. Crear el contexto.
2. Proveer el contexto:
Esto se logra envolviendo la componente 1 y 2 en el Provider.
3. Consumir el contexto:
Utilizaremos `useContext` para utilizar los valores y funciones `Setter`.



Context API

1. Crear el contexto

Para crear un contexto crearemos un archivo en la carpeta `src` llamado ***my_context.js***
Dentro, escribamos el siguiente código:

```
import { createContext } from "react"
const MyContext = createContext({})
export default MyContext
```

Esto nos permitirá disponibilizar, para nuestra aplicación, el contexto como un módulo que se pueda importar desde cualquier componente.

Context API

2. Proveer el contexto

```
import MyContext from "../my_context"
import Component1 from "../Component1"
import Component2 from "../Component2"

function App() {

  return (
    <>
      <MyContext.Provider value={/* Aquí
agregaremos el valor */}>
        <Component1 />
        <Component2 />
      </MyContext.Provider>
    </>
  )
}

export default App
```

Lo siguiente será modificar el código del archivo App.js para proveer un contexto.

1. Debemos importar el contexto creado.
2. Debemos envolver nuestra aplicación en el componente Provider **<MyContext.Provider>**

Context API

Anatomía de un provider

La anatomía de un provider es la siguiente:

```
<Contexto.Provider value={ <estado global> } >
```

- El nombre de la componente depende de cómo la exportemos cuando la creamos, en nuestro caso fue MyContext.
- El atributo **value** es el dato o conjunto de datos que estará disponible para los componentes envueltos por el contexto. Usualmente utilizaremos un objeto.

```
import MyContext from "./my_context"
import Component1 from "./Component1"
import Component2 from "./Component2"
import { useState } from "react"

function App() {
  const [data, setData] = useState(0)
  const sharedState = {data, setData}

  return (
    <>
      <MyContext.Provider value={ sharedState}>
        <Component1 />
        <Component2 />
      </MyContext.Provider>
    </>
  )
}

export default App
```

Context API

2. Proveer el contexto

- Lo que queremos compartir entre componentes es un estado, para eso utilizaremos **useState**.
-
- Debemos pasar el valor a compartir al componente **Provider**.

Context API

3. Consumir el contexto

```
/* Componente1.jsx */

import { useContext } from "react"
import MyContext from "../my_context"

const Component1 = () => {
  const { data, setData } = useContext(MyContext)
  return (
    <button onClick={e => setData(data + 1)}> +
  </button>
  );
}

export default Component1
```

- Para consumir el contexto, utilizamos useContext, es similar a useState solo que los valores los traemos del contexto en vez de ser locales al objeto.
- En el componente 1 aumentamos la cuenta.

Context API

3. Consumir el contexto

```
/* Componente2.jsx */

import MyContext from "../my_context"
import { useContext } from "react";

const Component2 = () => {
  const { data, setData } = useContext(MyContext)
  return (
    {value}
  )
}

export default Component2
```

- En el componente 2 mostramos la cuenta.

Ejercicio Modular Context

createContext + Provider

Existe una alternativa más limpia para trabajar tanto con createContext y el Provider.

Su beneficio es que así podemos modularizar y encapsular la lógica de los diferentes estados y métodos globales de nuestra aplicación.

Por lo tanto veamos un ejercicio sencillo:

Ejercicio Modular Context

createContext + Provider

- Vamos a realizar un ejercicio para compartir un “counter” (*contador*) entre diferentes componentes, **pero ahora lo llevaremos a un módulo** para organizar de mejor forma nuestro código.

Increment: 5

Decrement: 5

Ubicación del nuevo archivo:



Modularizar Context

CreateContext + Provider

```
/* src\context\CounterContext.jsx */
import { createContext, useState } from "react";

export const CounterContext = createContext();

const CounterProvider = ({ children }) => {
  const [counter, setCounter] = useState(0);
  return (
    <CounterContext.Provider value={{ counter, setCounter }}>
      {children}
    </CounterContext.Provider>
  );
};

export default CounterProvider;
```

1. Exporta "CounterContext" para que pueda ser utilizado en cualquier componente que necesite el contexto.
(utilizado en el hook useContext)
2. Crea un componente proveedor "CounterProvider" el cual tendrá tus estados o métodos globales.
3. El props "value" está compartiendo un objeto con el valor del estado y su respectivo setter.

Modularizar Context

```
import CounterProvider from "../context/CounterContext";

import Decrement from "../components/Decrement";
import Increment from "../components/Increment";

const App = () => {
  return (
    <div>
      <CounterProvider>
        <Increment />
        <Decrement />
      </CounterProvider>
    </div>
  );
};

export default App;
```



- Todos los componentes que desean acceder al contexto tienen que estar envueltos en nuestro "CounterProvider".
- En este caso estamos llamando a los componentes: "Increment" y "Decrement".

Modularizar Context

```
import { useContext } from "react";
import { CounterContext } from "../context/CounterContext";

const Increment = () => {
  const { counter, setCounter } = useContext(CounterContext);

  return (
    <button onClick={() => setCounter(counter + 1)}>
      Increment: {counter}
    </button>
  );
};

export default Increment;
```

Ahora utilizando el hook
“useContext” podemos acceder al
estado compartido por nuestro
proveedor.

Modularizar Context

Increment: 5

Decrement: 5

Intenta replicar el ejercicio anterior pero en el componente “Decrement”.

Verás que el estado es compartido entre estos componentes.

*/** Compartir estados entre componentes utilizando props. **/*



*/** Compartir estados entre componentes utilizando Context. **/*



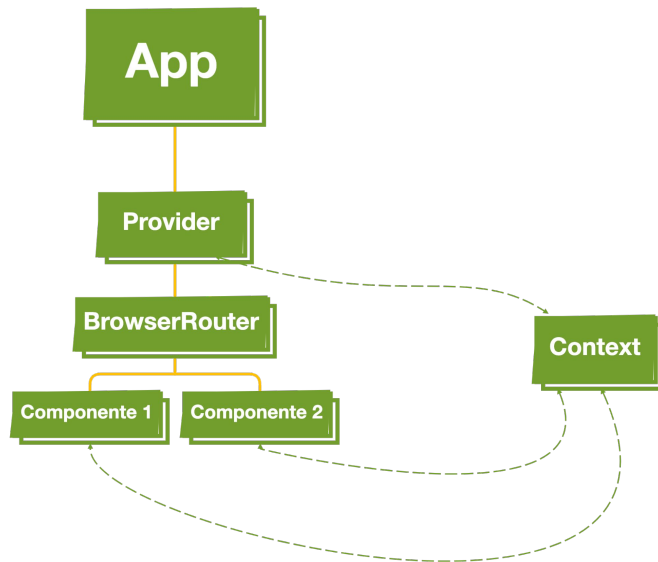
*/** Conocer el flujo de trabajo de React Router con Context para compartir información entre múltiples páginas. **/*

Objetivos

Contexto

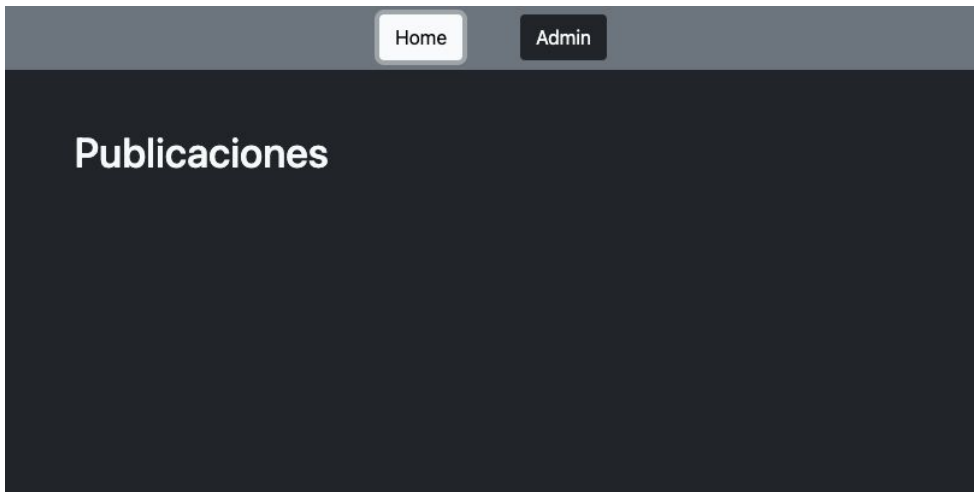
Contexto y rutas

- Trabajar con rutas y contextos no es más complejo y se repiten los mismos pasos.
- La componente Provider debe envolver el componente Browser Router.



Context API

Revisemos un proyecto llamado Planet Blog, el cual desde el administrador permite agregar publicaciones (posts) que se mostrarán en el home.



Tanto home como admin son componentes distintos, utilizando context podremos compartir las publicaciones agregadas dentro de Admin en el home.

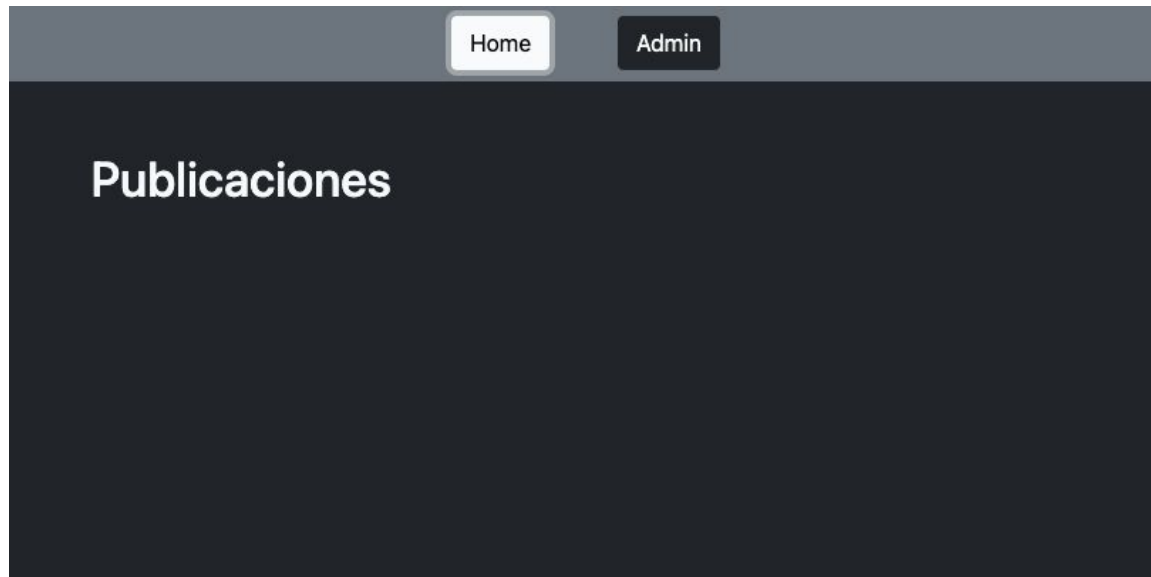
Context API

Setup del proyecto

1. Descarga el proyecto que se encuentra como material de apoyo en la sesión.
2. Dentro de la carpeta del proyecto ejecuta:
 - a. `npm install`
 - b. `npm run dev`
3. Dentro del navegador deberías ver el proyecto de la slide anterior corriendo.

Context API

Revisando la APP



Recordemos los pasos importantes de la implementación de un context, revisándolos en el proyecto.

1. Busquemos el contexto.
2. Busquemos el provider.
3. Busquemos los consumers.

Context API

1. Crear el contexto

```
import { createContext, useState } from "react";

export const PostsContext = createContext();

const PostProvider = ({ children }) => {
  const [posts, setPosts] = useState([]);

  return (
    <PostsContext.Provider
      value={{
        posts,
        setPosts,
      }}
    >
      {children}
    </PostsContext.Provider>
  );
};

export default PostProvider;
```

Si revisamos el código veremos que no hay mayor diferencia en el contexto creado salvo el nombre.

Además, creamos un estado el cual será compartido a toda nuestra aplicación.

Recuerda que el “value” comparte estos datos como un objeto.

Context API

2. Proveer el contexto

```
import PostsProvider from "../context/PostsContext";

const App = () => {
  return (
    <PostsProvider>
      <Navbar />
      <main className="container">
        <Routes>
          <Route
            path="/"
            element={<Home />}
          />
          <Route
            path="/admin"
            element={<Admin />}
          />
        </Routes>
      </main>
    </PostsProvider>
  );
};

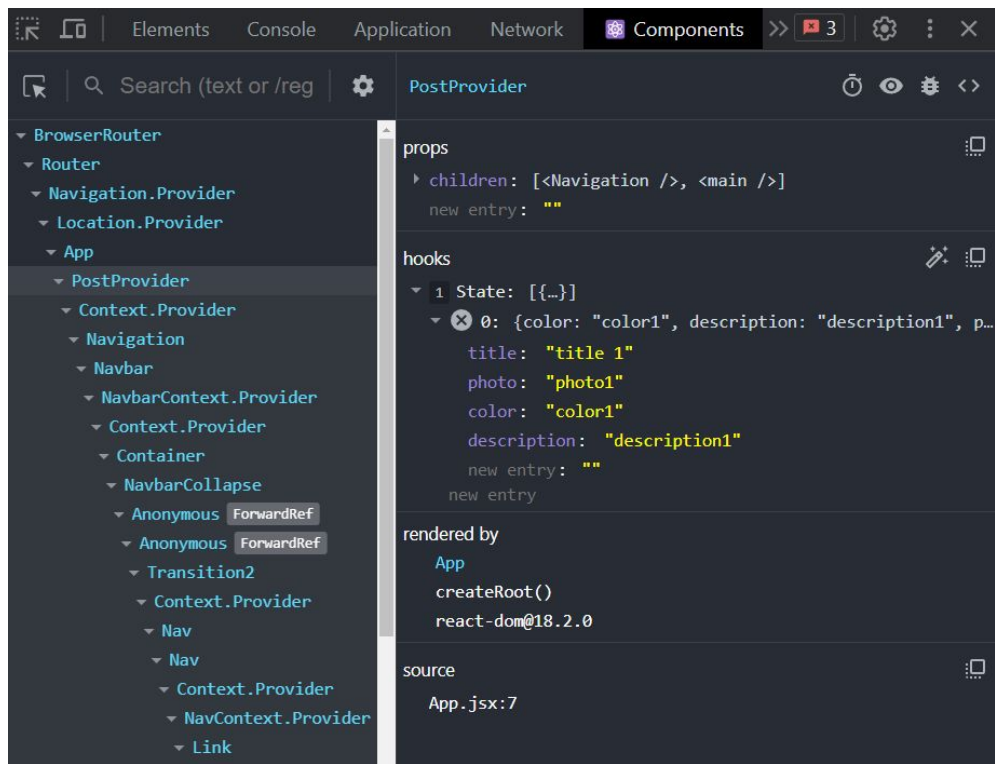
export default App;
```

Proveer un contexto

1. Se importa el PostProvider
2. Se envuelve la aplicación en el componente Provider
<PostsProvider>

Context API

El Provider



Si necesitáramos revisar que nuestro contexto y estado global se crearon correctamente, podemos ocupar React Dev Tools y revisar que nuestro Provider está incluido en el árbol de componentes.

Context API

3. El Consumer

Cualquier componente dentro de un componente provider puede ser consumer si se utiliza el hook `useContext`.

Revisemos en el archivo *PostCreate.jsx*, la importación del contexto que creamos anteriormente y el *useState*.

```
import { PostsContext } from "../context/PostsContext";
```

Ahora utilizando el hook `useContext` se pueden recuperar las publicaciones

```
const { posts, setPosts } = useContext(PostsContext);
```

Context API

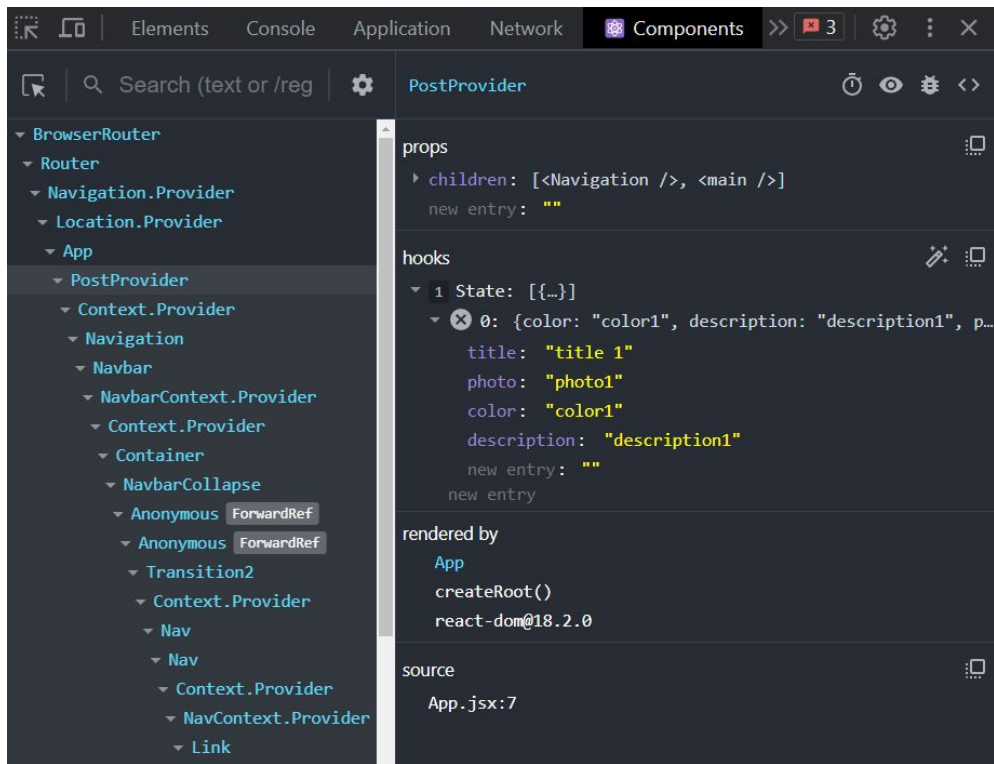
El hook useContext

La función **handleSubmit** agrega una nueva publicación utilizando la función setter del contexto, de esta forma podemos compartir el valor.

```
const handleSubmit = (e) => {  
  e.preventDefault();  
  setPosts([...posts, post]);  
  alert("Publicación agregada");  
};
```

Context API

El hook useContext



Ahora probemos el formulario de nuestra aplicación agregando una nueva publicación y verificando en el React Dev Tools que efectivamente fue agregada en el estado global.

Context API

Renderización dinámica con datos del estado global

Ahora que ya revisamos la creación de publicaciones, revisemos la vista Home:

```
import { useContext } from "react";  
import { PostsContext } from "../context/PostsContext";  
import PostCard from "../components/PostCard";
```

Context API

Renderización dinámica

```
const Home = () => {  
  const { posts } = useContext(PostsContext);  
  
  return (  
    <>  
      <h1 className="pb-3">Publicaciones</h1>  
      <div className="publicaciones row">  
        {posts.map((post, i) => (  
          <div  
            className="col-12 col-sm-6 col-md-4"  
            key={i}  
          >  
            <PostCard post={post} />  
          </div>  
        ))}  
      </div>  
    </>  
  );  
};  
  
export default Home;
```

Finalmente, revisemos el código que muestra las publicaciones y veremos que no es distinto a ocupar estados, solo que traemos los valores del contexto.

*/** Compartir estados entre componentes utilizando props. **/*



*/** Compartir estados entre componentes utilizando Context. **/*



*/** Conocer el flujo de trabajo de React Router con Context para compartir información entre múltiples páginas. **/*



Objetivos

Ideas importantes:

- Context nos permite compartir estados, pero no es la única forma. Si solo necesitamos compartir el estado entre los componentes padre e hijo, podemos pasar los estados como props.
- Podemos tener múltiples contextos, solo necesitan tener nombres distintos. Sin embargo, si son los mismos componentes los que los van a consumir, es mejor agrupar los datos en un único contexto.
- Context nos permite resolver el problema de prop drilling.
- Context hace más complejo mantener los componentes independientemente, usar sin abusar.



Reflexionemos



Cierre

{desafío}
latam_



¿Existe algún concepto que no
hayas comprendido?

Reflexionemos

- Revisemos el material de estudio asincrónico en donde repasamos los conceptos de esta clase.
- Revisemos el desafío de la unidad leyendo la descripción y requerimientos.

¿Qué sigue?

¿Tienen alguna duda respecto al Desafío?



*Academia de
talentos digitales*

www.desafiolatam.com



/DesafioLatam



/DesafioLatam



/DesafioLatam



/DesafioLatam